Fig. 7: Device Manager window

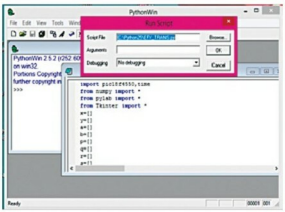

Fig. 9: Opening efy_trans.py file and running it using pywin32

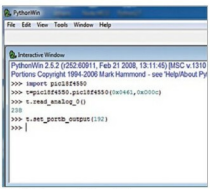

Fig. 8: Interactive Python shell

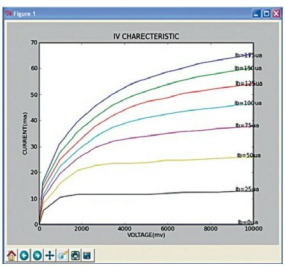

Fig. 10: Program output of IV characteristic curve of transistor

descriptors contain information about the USB device and type of communication. If you want to design your own USB device, be aware of the endpoint details, vender IDs and product IDs. This information is necessary for device enumeration and is given in usb_desc_cdc1.h file. The firmware, in addition to USB requirement, implements the following tasks:

- Generates different base voltages
- Generates different emitter currents
- Measures collector voltages
- Sends data to PC

The USB host communicates with the USB peripheral through the device driver. The device driver is a software component that enables an application to access a hardware device. USB device drivers for Windows must conform to Win32 driver model.

Windows includes application programmer's interface functions that enable applications to communicate with device drivers. The device driver is generated using Library USB Wizard, and the device is installed in the host computer. Libusb-win32 is a port of the USB library for Windows operating system. The library allows user space applications to access any USB device on Windows in a generic way without writing any line of kernel driver code.

Hardware installation procedure

Step 1. After fusing the program (hex file) into the microcontroller, connect the board to the PC. As soon as the two are connected, a pop-up screen appears, as shown in Fig. 6.

Step 2. Use inf- wizard found in libusb-win32-device-bin-1.2.6.0 folder to generate the driver for the attached device. If the driver is installed properly, the attached device will appear in Device Manager, as shown in Fig. 7.

Step 3. Basic functions including USB configuration, setting port B bits and reading analogue voltages can be tested using default interactive Python window. Using set_portb_ output (192) command, connected LEDs (LED1 and LED2) can be lit.

Similarly, analogue voltages can be measured by connecting the terminal (AN0) to 5V. The value corresponding to 5V is 255. It is better to test the basic function using interactive Python window (Fig. 8) before testing the actual board.

Step 4. Open efy_trans.py file from PythonWin 2.5.2 Interactive Window and run the script as shown in Fig. 9. The program output pattern is displayed using Tkinter software, which is shown in Fig. 10.

The USB based data-acquisition system using Python interface and open source general-purpose device driver reduces the complexity involved in USB connectivity. Interactive nature of Python software makes USB connectivity more user-friendly as compared to other visual languages. EFY

EFY Note

The source code of this project is included in this month's EFY DVD and is also available for free download at *source.efymag.com*

A. Robson Benjamin is associate professor of Physics at American College, Madurai, Tamil Nadu. His interests include design and development of USB based data acquisition and automation of electronic gadgets